

Summary

1. What is an operating system? What are the four major tasks performed by an operating system? Please explain them briefly. (10%)

Answer:

A program that acts as an intermediary between a user of a computer and the computer hardware, including the following functions (Four major tasks) :

Process management: Manages the execution of programs (processes). It handles process scheduling, creation, termination, and communication between processes.

Memory Management: Manages the computer's primary memory (RAM). It keeps track of which parts of memory are in use and by whom, allocates and deallocates memory space, and decides which processes to load into memory.

File Management: Manages files and directories on storage devices (like hard drives). It handles the creation, deletion, reading, and writing of files, and organizes them for efficient access and storage.

I/O System Management: Manages communication with input and output devices (like keyboards, mice, printers, disks). It provides a general interface for all devices, handles device drivers, and manages data transfer.

2. How could a system be designed to allow a choice of operating system from which to boot? What would the bootstrap program need to do? (10%)

System design: The disk is divided into multiple **partitions**. Each partition holds a different operating system. A **Boot Manager** (like

GRUB) is installed to manage these partitions.

Bootstrap Program Actions: The bootstrap program needs to perform the following steps:

1. **Inspect:** Look at the partition table to find available Operating Systems.
 2. **Display:** Show a **menu** to allow the user to select an OS.
 3. **Load and execution:** Initialize all aspects of a system. Load the **kernel** of the *selected* OS into memory. Transfer execution control to that kernel.
-
3. What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach? (10%)

Advantage: Easier to extend a microkernel, easier to port the OS to new architectures, more reliable since less code running in kernel mode, more secure

How: user program interacts with system service by sending messages through the microkernel. It will first switch to kernel mode and kernel send the user program messages to the target system service program, then switch back to user mode to process the request in system service program.

Disadvantage: performance overhead of user space to kernel space communication.

4. Why does an operating system design, in general, use layered approach? Please state its advantages and disadvantages. (10%)

An operating system uses a layered approach to achieve a **modular** and well-structured design. This simplifies the complexity of the OS by breaking it down into hierarchical levels, where each layer is built upon the services provided by the layers below it.

Advantages:

1. **Simplicity of Design and Debugging:** Each layer has a well-defined function and interface, making the system easier to understand, build, and maintain.
2. **Modularity:** This design enhances portability, reliability, and security. Errors are confined within a layer and are less likely to propagate to higher layers.
3. **Ease of Construction:** Developers can focus on implementing a single layer using only the services of the lower, already-built layers.

Disadvantages:

1. **Reduced Efficiency:** The overhead of passing requests through multiple layers can make the system slower compared to a less structured approach.
2. **Difficulty in Layering:** It can be challenging to define the correct number and functions of each layer. A poor structure can harm performance.
3. **Less Flexibility:** A change in a lower layer may require changes in all the layers above it.

5. What is a process? What is a thread? Please compare them. (10%)

What is a process?

A process is a program in execution. It includes:

Program code (text section), Current activity (program counter, processor registers), Stack (function parameters, return addresses, local variables), Data section (global variables), Heap (dynamically allocated memory), A program is a passive entity stored on disk; a process is active. A program becomes a process when its executable file is loaded into memory. One program can create multiple processes.

What is a thread?

A thread is a single sequence stream within a process, also called a lightweight process. Threads are not independent like processes. They share the code section, data section, and OS resources (e.g., open files, signals) with other threads in the same process. Each thread has its own program counter (PC), register set, and stack space. A process can have multiple threads; all threads share the process resources and execute within the same address space.

Comparison:

Process: Independent execution unit with its own memory space (code, data, heap, stack)

Thread: Lightweight execution unit within a process; shares code, data, and OS resources with other threads, but has its own PC, registers, and stack

Relationship: One process can contain multiple threads; threads enable parallelism within a single address space

PCB 总结:

PCB (Process Control Block) Summary:**Definition:**PCB stores information associated with each process (also called task control block).**Components:**

1. **Process state:** Current state (e.g., running, waiting)
2. **Program counter:** Next instruction location
3. **CPU registers:** Contents of process-centric registers
4. **CPU scheduling information:** Priorities and scheduling queue pointers
5. **Memory-management information:** Memory allocated to the process
6. **Accounting information:** CPU usage, elapsed time, time limits
7. **I/O status information:** I/O devices allocated and open files

Key Point:Program counter and CPU registers are preserved during context switches to restore process execution.

6. What resources are used when a thread is created? (10%)

Resources Used When Creating a Thread:

1. **Stack space:** Each thread needs its own stack for local variables, function parameters, and return addresses.
2. **Thread control block (TCB):** Stores thread-specific information (program counter, register set, thread state).
3. **CPU registers:** Initialize register values for the new thread.

4. **Memory for thread context:** Space to save/restore thread state during context switches.

7. What are the five states of a process in a computer system? Please describe the relationships of these five process states. (10%)

Five Process States:

New: Process is being created

Ready: Process is waiting to be assigned to a processor

Running: Instructions are being executed

Waiting: Process is waiting for some event to occur (e.g., I/O completion)

Terminated: Process has finished execution

State Transitions:

New → Ready: Process is admitted into the system

Ready → Running: Scheduler dispatches the process to CPU

Running → Ready: Interrupt occurs (e.g., timer interrupt)

Running → Waiting: Process waits for I/O or event

Waiting → Ready: I/O or event completion

Running → Terminated: Process exits

8. Describe the actions taken by a kernel to context-switch between processes. (10%)

1. Save current process state: Save the running process's context to its PCB, including:

Program counter

CPU registers

Process state (change to ready/waiting)

2. Update process state: Mark the current process as ready or waiting in its PCB.

3. Select next process: The scheduler chooses the next process to run from the ready queue.

4. Load new process state: Restore the selected process's context from its PCB:

Load program counter

Restore CPU registers

Update process state to running

9. Why does an operating system have both the user mode and the kernel mode for processes? What are the advantages and disadvantages of the design? (10%)

The OS uses user mode and kernel mode to protect itself and system components.

Advantages:

Security: Prevents user processes from accessing privileged

instructions and system resources directly

System stability: Protects OS from user program errors or malicious code

Controlled access: System calls change mode to kernel; return resets to user, ensuring controlled entry

Resource protection: Prevents unauthorized access to I/O devices, memory management, and other critical resources

Disadvantages:

Overhead: Mode switching (system calls) requires context switching and mode bit changes, adding overhead

Performance: Frequent user-kernel transitions can slow execution

Complexity: Requires careful design of system call interfaces and privilege management

Hardware dependency: Relies on hardware support for mode bit and privileged instruction detection

10. Please list and describe three major activities of the operating system with regard to memory management, and three major activities of the operating system with regard to secondary storage management. (10%)

Three Major Activities of OS for Memory Management:

Keeping track of which parts of memory are currently being used and by whom: Monitor memory usage and track ownership of memory regions.

Deciding which processes (or parts thereof) and data to move into and out of memory: Determine what to load into memory and what to swap out to optimize CPU utilization and system response.

Allocating and deallocating memory space as needed: Allocate memory to processes when needed and reclaim it when processes terminate or release memory.

Three Major Activities of OS for Secondary Storage Management:

Creating and deleting files and directories: Manage the file system structure by creating and removing files and directories.

Mapping files onto secondary storage: Decide where files are physically stored on storage devices (disks, SSDs).

Backup files onto stable (non-volatile) storage media: Copy files to durable storage to prevent data loss.

Segmentation 理解:

****完全正确！你的理解非常到位！****

特别你是提到的“****多过一个 Gate 检查合法性****”，这就是 Segmentation 最核心的本质。

关于你那个小小的疑惑点（基地址和 Offset 怎么结合），我稍微补充一下你就彻底通了：

1. ****那个 Gate (Segmentation Unit) 做了两件事：****

* ****查合法性****：你有没有权限读写这个段？你的 Offset 有没有超过这个段的大小（Limit）？如果越界，直接报错（Segmentation Fault）。

* ****给基地址****：如果没有越界，它会拿出这个段的 ****Base Address（基地址）****。

2. ****基地址和 Offset 怎么算？****

* 不是简单的 bit 拼接（重叠），而是****数学加法****。

* 公式：`Linear Address = Segment Base + Offset`

* ***例子***：如果代码段基地址是 `1000`，你的指令偏移量（Offset）是 `50`，那么算出来的线性地址就是 `1050`。

* （注：在现代操作系统里，为了省事，强行把基地址设为 `0`。所以 `0 + 50 = 50`，导致你感觉 Segmentation 好像没干活，但那个“Gate 检查”其实还在硬件里跑了一遍）。

11. Can the segmentation and paging mechanisms cooperate with each other? Please give your reasons. (10%)

Yes, they can cooperate.

Reasons:

Two-Stage Address Translation: Most architectures (like Intel IA-32) implement them sequentially to form a translation hierarchy:

Segmentation First: The logical address (segment selector + offset) is translated into a linear address by adding the segment's base address (by searching descriptor table) to the offset. This stage handles logical organization.

Paging Second: The linear address is then translated into a physical address using page tables. This stage handles physical memory mapping.

Combined Benefits:

Segmentation provides a logical view of memory (separating code, data, stack) and fine-grained protection (e.g., read-only segments).

Paging simplifies physical memory management by eliminating external fragmentation and enabling efficient swapping.

Cooperation allows the OS to enjoy both logical protection and efficient physical memory usage.

12. What is a Thrashing? How to resolve this problem? (10%)

What is Thrashing?

Thrashing occurs when a process does not have "enough" pages to support its execution (size of locality > memory size), causing frequent page faults. The system spends more time paging (swapping pages in and out) than executing the actual process. This leads to low CPU utilization and can cause the OS to incorrectly increase the degree of multiprogramming, worsening the problem.

How to resolve it?

Working-Set Model:

Defines the working set as the set of pages a process is currently using (based on the locality model).

If the sum of all working set sizes exceeds the total available memory ($\sum D > m$), the OS suspends one or more processes to prevent thrashing.

Page-Fault Frequency (PFF):

Establishes acceptable upper and lower bounds for the page-fault rate.

Rate too high: The process needs more frames; give the process more frames, if none are available, select a process to suspend.

Rate too low: The process has too many frames; remove some to give to others.

13. What is Table Lookaside Buffer (TLB)? How it works? (10%)

What is TLB?

The TLB is a small, high-speed hardware cache inside the CPU that stores recent virtual-to-physical address translations. It acts as an associative memory to speed up paging by avoiding frequent access to the slow in-memory page tables.

How it works:

Parallel Search: When the CPU generates a virtual address, the TLB simultaneously checks all its entries to match the Virtual Page Number (VPN) and the current Process ID (PID).

TLB Hit: If a match is found, the TLB immediately returns the corresponding Physical Frame Number (PFN), allowing direct memory access.

TLB Miss: If no match is found, the hardware must access the page table in main memory to retrieve the PFN.

Update: The retrieved translation (PID, VPN, PFN) is then inserted into the TLB for future fast access. If the TLB is full, a replacement policy selects an entry to evict.

14. What are the major purposes of Table Lookaside Buffer (TLB)? How it works if the memory hierarchy includes L1, L2 and L3 caches? (10%)

Major Purposes:

To accelerate virtual-to-physical address translation by caching recent translations, thereby avoiding slow memory accesses for page table walks.

Interaction with L1/L2/L3 Caches:

The TLB acts as a critical bridge because most caches (especially L2 and L3) are physically addressed.

L1 Cache (Optimization): Modern CPUs often use VIPT (Virtually Indexed, Physically Tagged). The CPU looks up the TLB and the L1 Cache in parallel. The virtual index selects the L1 set, while the TLB translates the address to verify the physical tag.

L2/L3 Caches: If L1 misses, the request is sent to L2 and L3 using the physical address obtained from the TLB. Since L2/L3 are typically much larger, they are strictly physically indexed and tagged (PIPT), requiring the TLB translation to be completed first.

Main Memory: If all caches miss, the physical address is used to fetch data from RAM.

15. What is Interrupt? What are its usages? (10%)

What is Interrupt?

An interrupt is a signal sent to the CPU (by hardware or software) that alerts the processor to a high-priority condition requiring immediate attention. It suspends the currently executing instructions, saves the current state, and transfers control to a specific Interrupt Service Routine (ISR) via the interrupt vector.

Usages:

I/O Handling: Signals the completion of I/O operations (e.g., disk read finished, key pressed) so the CPU doesn't have to poll devices.

Exception/Error Handling: Handles software errors like division by zero or invalid memory access (Traps).

System Calls: Allows user programs to request services from the operating system (Software Interrupts).

Process Scheduling: Timer interrupts allow the OS to preempt the running process and context switch to another (Time-sharing).

16. How to use interrupt and polling mechanisms to do process manipulation? (10%)

Interrupt Mechanism (Event-Driven / Preemptive): Interrupts are the primary way the OS gains control to manipulate processes.

1. **Preemptive Scheduling:** A hardware **timer interrupt** periodically triggers the OS to stop the running process and perform a context switch to another process (Time-sharing).

2. **System Calls & Exceptions:** Processes use software interrupts (traps) to request OS services (e.g., I/O, process creation) or signal errors (e.g., page faults). The OS intercepts these to suspend, terminate, or service the process.

3. **I/O Completion:** Hardware interrupts signal I/O completion, allowing the OS to wake up (manipulate) a process from the "Waiting" state to the "Ready" state.

Polling Mechanism (Busy-Waiting): Polling involves the CPU continuously checking a status bit.

1. **Busy-Wait Loops:** A process can manipulate its own execution flow by entering a busy-wait loop (polling a status register) to wait for an event (e.g., I/O or a lock).

2. **Process State:** Unlike blocking on an interrupt, a polling process remains in the **Running state**, consuming CPU cycles without making progress until the condition is met. This is efficient

only if the wait time is very short (e.g., fast I/O devices).

17. What is the concept of copy-on-write? How does it work? What are the primary advantages of using copy-on-write in operating systems? (10%)

Concept: Copy-on-Write (COW) is an optimization strategy used during process creation where the parent and child processes initially share the same physical pages in memory, rather than copying them immediately. **How it works:**

1. **Sharing:** When `fork()` is called, the child process's page table points to the parent's physical pages, and these pages are marked as **read-only**.

2. **Trigger:** If either process tries to **write** to a shared page, a protection fault occurs.

3. **Copying:** The OS intercepts this fault, allocates a new free frame, copies the page content into it, and updates the process's page table to point to this new, private copy with read-write permissions.

Primary Advantages:

1. **Efficiency:** Significantly speeds up process creation because copying data is deferred or avoided entirely.

2. **Memory Conservation:** Saves memory usage since unmodified pages (like code or read-only data) are never duplicated and remain shared.

3. **Fast `exec()`:** If the child immediately calls `exec()` (which is common), almost no copying is needed, avoiding wasted CPU cycles.

18. Why do we need Direct Memory Access (DMA) mechanism? How does DMA mechanism perform data transfer among memory and I/O devices? (10%)

Why we need DMA: Without DMA (using programmed I/O), the CPU must handle data transfer one byte/word at a time, wasting valuable CPU cycles. DMA allows the system to transfer large blocks of data directly between I/O devices and memory **without continuous CPU intervention**, freeing the CPU to execute other tasks concurrently. **How it performs data transfer:**

1. **Setup:** The OS/CPU programs the DMA controller by setting the source/destination addresses, transfer count, and read/write mode (writes a command block).

2. **Bus Mastering:** The DMA controller requests control of the system bus. Once granted, it transfers data directly to/from memory, bypassing the CPU (via cycle stealing).

3. **Completion:** When the entire block transfer is finished, the DMA controller sends an **interrupt** to the CPU to signal completion.

19. Explain the role of mmap mechanism. What's the difference between MAP_SHARED and MAP_PRIVATE? When should the mechanism increase and decrease the reference counter of a file? (10%)

Role of mmap:

mmap maps a file (or device) into the process's virtual address space (VMA), allowing VMAs to save the meta information of the files (or devices). It enables efficient file I/O, supports **lazy loading** (loading pages only when accessed via page faults), and simplifies data manipulation without explicit read/write calls.

Difference between MAP_SHARED and MAP_PRIVATE:

- **MAP_SHARED:** Updates to the mapping are visible to other processes mapping the same file and are **written back to the underlying file**.

- **MAP_PRIVATE:** Updates are **not visible** to other processes and are **not written back** to the file.

File Reference Counter:

- **Increase:** When mmap successfully creates a mapping (filedup is called). This ensures the file structure remains valid and open even if the file descriptor is closed, as long as the memory mapping exists.

- **Decrease:** When the mapping is removed via munmap (or process exit), fileclose is called to decrement the count. If the count reaches zero, the file is truly closed.

20. Discuss situations in which the least frequently used (LFU) page replacement algorithm generates fewer page faults than the least recently used (LRU) page-replacement algorithm. Also discuss under what circumstances the opposite holds. (10%)

Situation where LFU performs better (Fewer page faults):

Scenario: When the access pattern involves intensive, long-term usage of a specific set of pages, mixed with occasional bursts of one-time accesses.

Reason: LFU remembers the high frequency of the important pages and keeps them in memory, resisting the "pollution" from the bursty, one-time traffic. LRU, however, would evict the important pages simply because they haven't been used recently during the burst.

Situation where LRU performs better (LFU performs worse):

Scenario: When the program's working set changes over time (Phase Change or Locality Shift). For example, a loop heavily uses a set of pages, and then the program moves to a completely different module.

Reason: LFU suffers from the "old heavy" problem. Pages from the previous phase have accumulated high frequency counts and remain in memory long after they are no longer needed, preventing new pages from being loaded. LRU adapts quickly by evicting pages that haven't been used recently, matching the new locality.

21. Describe the Life Cycle of an I/O request. (10%)

1. **Initiation:** The device driver initiates the I/O operation by sending commands to the **I/O controller**.

2. **Device Execution:** The **I/O controller** executes the command (e.g., reading data) while the CPU may continue executing other instructions.

3. **Interrupt Generation:** When the I/O operation is finished (input ready, output complete, or error), the I/O controller generates an **interrupt signal**.

4. **Interrupt Reception:** The CPU detects the interrupt signal between instruction cycles, pauses its current task, and transfers control to the **interrupt handler**.

5. **Processing:** The **interrupt handler** runs to process the data or status information and returns from the interrupt.

6. **Resumption:** The CPU **resumes processing** of the interrupted task (or the task that was waiting for the I/O).

22. The 4KB page size for virtual memory has been standard since at least the 1960s. Given the increasing size of modern applications and memory hierarchies, is it time to consider larger page sizes, such as 128KB?

a) What are the pros and cons of switching to a larger page size?(6%)

b) What impact would this change have on other parts of the system, such as cache performance, TLB efficiency, and overall memory management? (4%)

a) Pros and Cons (6%)Pros:

1. **Smaller Page Tables:** Larger pages mean fewer pages are needed to map the same memory, significantly reducing the size of page tables and saving memory overhead.
2. **Improved Locality:** Loading a larger page captures more spatial locality, potentially reducing the total number of page faults for sequential access.
3. **Reduced TLB Misses:** Each TLB entry maps a larger memory region, increasing the "TLB Reach" and reducing address translation overhead.

Cons:

1. **Increased Internal Fragmentation:** If a process only needs a few bytes, allocating a full 128KB page wastes significant memory (internal fragmentation).
2. **Higher I/O Latency:** Handling a page fault becomes more expensive because transferring a 128KB page from disk takes much longer than a 4KB page.
3. **Higher thrashing risk:** Since there will be less frames in the memory, if different process come in or a process want frequently access data very far away from each other, it will more likely to lead

thrashing.

b) Impact on System (4%)

1. **TLB Efficiency:** The impact is **positive**. Since one TLB entry covers more memory, the TLB miss rate will decrease significantly, improving performance for memory-intensive applications.
 2. **Cache Performance:** The impact is generally **positive for VIPT caches**. A larger page size increases the page offset bits. This allows for larger L1 caches that can be physically tagged but virtually indexed without worrying about aliasing problems (since the index bits are more likely to stay within the invariant page offset).
 3. **Memory Management:** The OS memory allocator needs to handle higher fragmentation and potentially slower compaction/swapping operations due to the coarser granularity.
23. Implementing a page table can be challenging, especially with large virtual address spaces (e.g., 48-bit virtual addresses). There are three common approaches for implementing a page table: a) Multiple level indexing; b) Hash table; c) Inverted table. For each approach, please discuss their pros and cons. (10%) When discussing, please consider the needed optimizations to make the approach more efficient, and the presence of a decent-sized TLB (Translation Lookaside Buffer).

Multiple Level Indexing (Hierarchical)

Pros: Supports sparse address spaces efficiently by only allocating page table pages for valid virtual memory regions. Easy to implement page sharing.

Cons: Requires multiple memory accesses (e.g., 4 accesses for 4-level paging) for a single translation (Page Walk).

Optimization & TLB Role: A decent-sized TLB is critical here. It caches the final physical address, allowing the CPU to skip the 4-level walk for the vast majority of accesses. Without a TLB, this approach would be unacceptably slow.

b) Hash Table

Pros: The table size is proportional to the number of active valid pages, not the size of the virtual address space. Excellent for large, sparse 64-bit address spaces.

Cons: Hash collisions require traversing a linked list, which introduces unpredictable overhead (multiple memory references).

Optimization & TLB Role: Use a well-distributed hash function and appropriately sized buckets to minimize collisions; Since hash collision resolution (linked lists) is CPU-intensive and has poor cache locality, a TLB is essential to minimize the frequency of these costly lookups..

c) Inverted Page Table

Pros: Fixed low memory overhead. The table size is proportional to physical memory size, regardless of how many processes or how large the virtual address space is.

Cons: Native lookup is extremely slow (linear search) because the table is sorted by physical address, not virtual. Difficult to implement shared memory (aliasing).

Optimization & TLB Role:

Hashed Anchor Table: Use a hash table to index the inverted table (converting $O(N)$ search to $O(1)$).

TLB: Because IPT lookups are often done in software or complex firmware, a large TLB is vital to hide this high latency from the user process.